

Chapter 4:

Machine Learning



Basic concepts

Machine learning

In the data science application of machine learning methods, machine learning can be seen as a means of

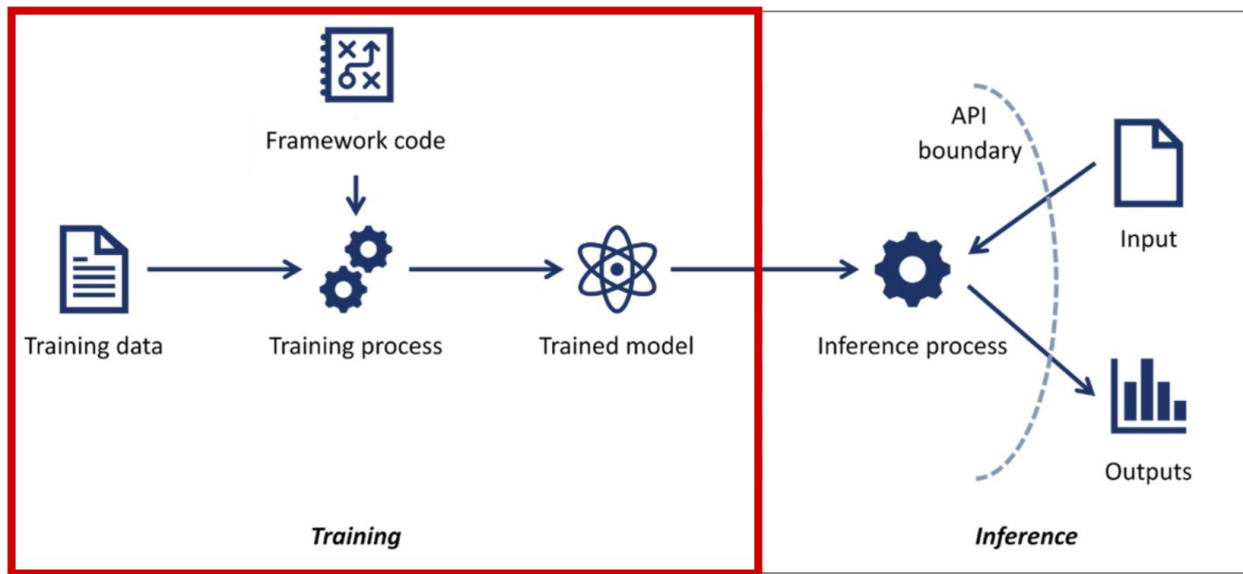
- building models to help understand existing data
- using these models to predict unseen data

Categories of Machine Learning

- At the most fundamental level, machine learning can be categorized into 4 main types:
 - supervised learning
 - unsupervised learning.
 - semi-supervised learning
 - reinforcement learning

Categories of Machine Learning

- Supervised learning involves modeling the relationship between features of data and some label associated with the data; once this model is determined, it can be used to apply labels to new, unknown data.
 - classification, the labels are discrete categories,
 - regression, the labels are continuous quantities.



Categories of Machine Learning

- Unsupervised learning involves modeling the features of a dataset without reference to any label. Two common unsupervised tasks includes
 - Clustering algorithms identify distinct groups of data
 - Dimensionality reduction algorithms search for more brief representations of the data.

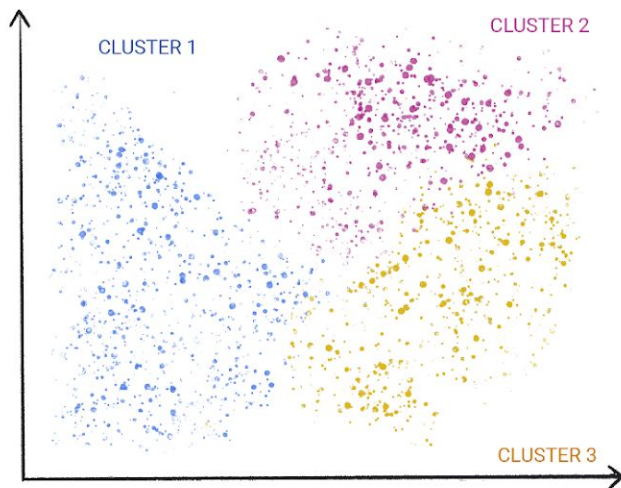


Figure 1. An ML model clustering similar data points.

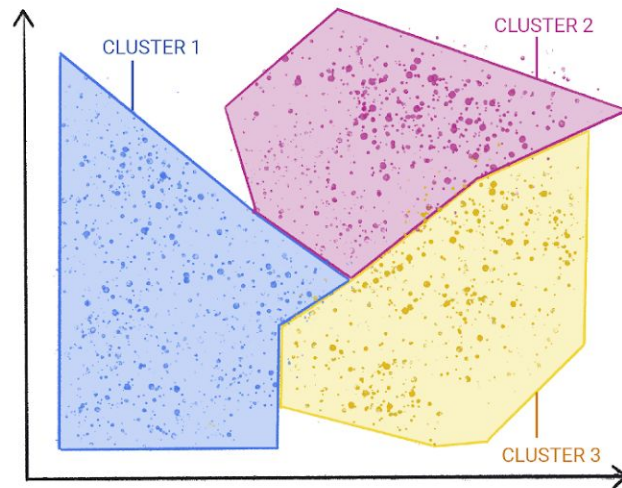


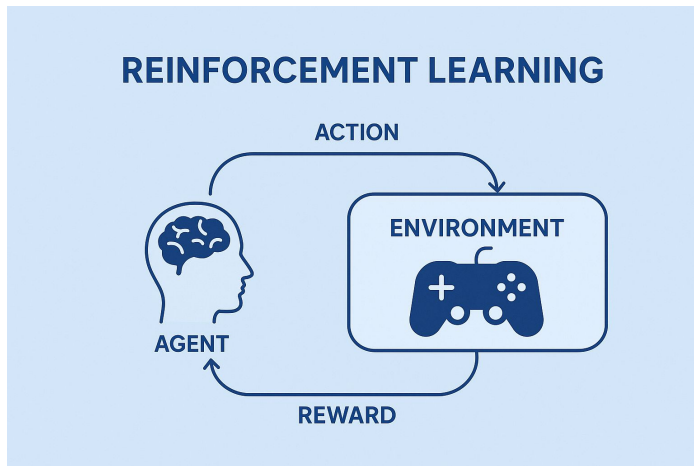
Figure 2. Groups of clusters with natural demarcations.

Categories of Machine Learning

- Semi-supervised learning methods combine supervised and unsupervised learning by using both labeled and unlabeled data to build models

Categories of Machine Learning

- The agent observes the current state of the environment.
- Based on its policy, the agent selects and performs an action.
- The environment responds to the action, transitioning to a new state.
- The agent receives a reward signal, reflecting the value of the new state.
- This reward information is used to update the agent's policy, making it more likely to choose actions that led to positive rewards in the past.



Model evaluation

- Holdout sets:
 - hold back some subset of the data from the training of the model, and then use this holdout set to check the model performance.
- Cross-validation
 - if the initial set of training data is small, do a sequence of fits where each subset of the data is used both as a training set and as a validation set.

Model evaluation

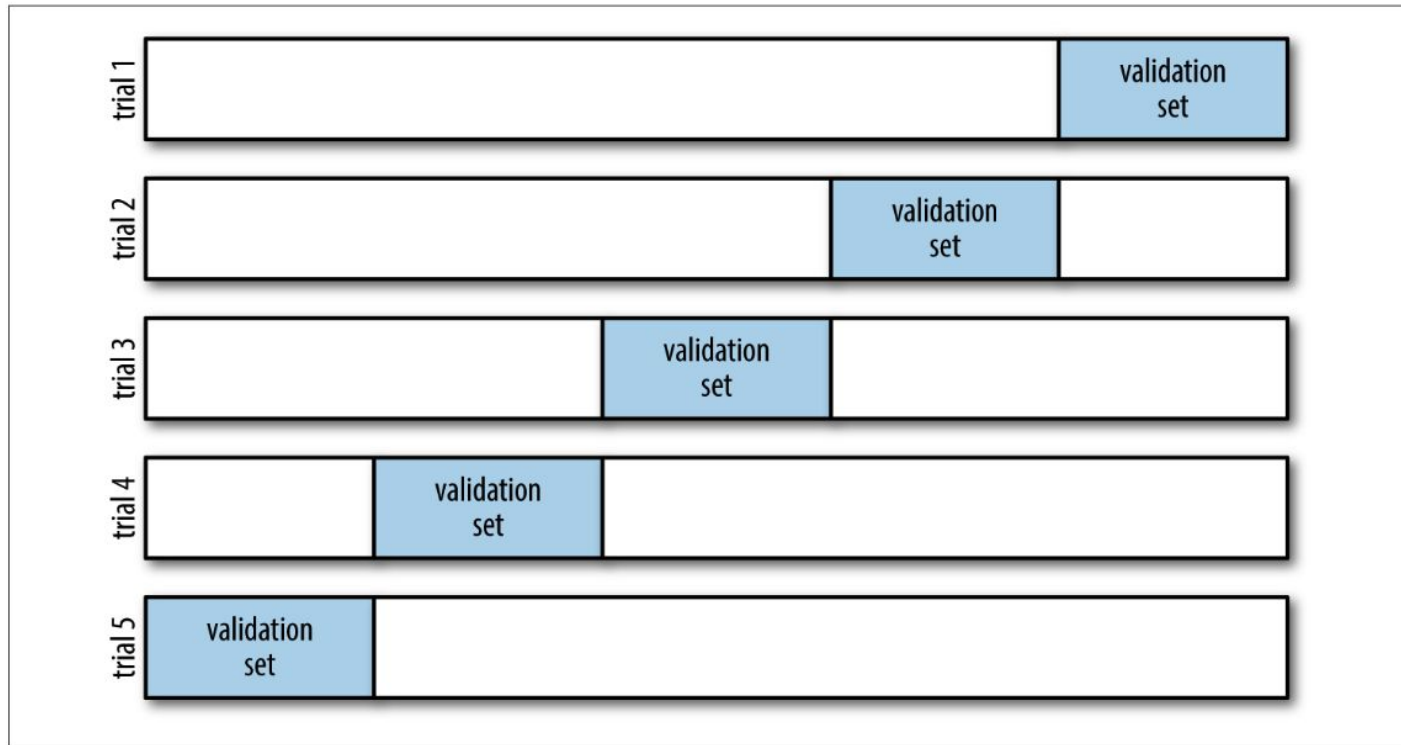


Figure 5-23. Visualization of five-fold cross-validation

Overfitting and underfitting

- Overfitting: producing a model that performs well on the data you train it on but that generalizes poorly to any new data.
 - Simple model
 - Small dataset

Overfitting and underfitting

- Underfitting: produce a model that doesn't perform well even on the training data, although typically when this happens you decide your model isn't good enough and keep looking for a better one.

Bias

- Bias: the error between **the average predicted value** of the model and **the actual value**.

$$Bias = E[\hat{f}(x)] - f(x)$$

- High bias: the model underfits, cannot capture complexity.
- Low bias: the model is flexible enough to represent the data well.

Variance

- Variance: measure how much a model's predictions would change if we trained it on different training sets
 - high variance: the model's predictions fluctuate a lot depending on the dataset.
 - low variance: predictions are stable across different training sets.

Selecting the Best Model

Bias vs Variance

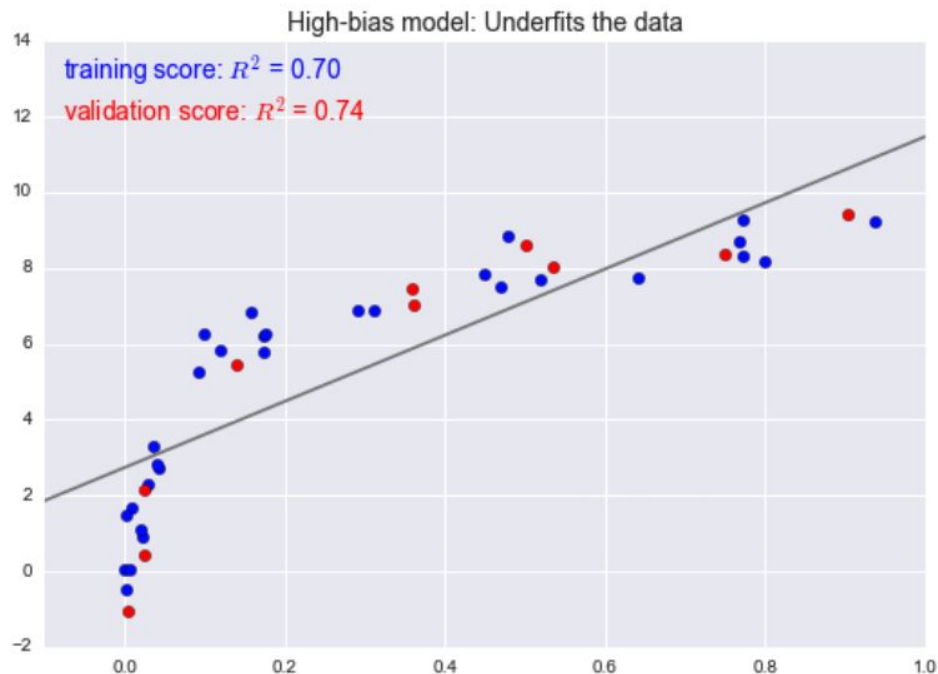
- Bias: error from wrong assumptions (too simple model).
- Variance: error from too much sensitivity to training data (too complex model).

Selecting the Best Model: finding the right bias–variance tradeoff.

Selecting the Best Model

The bias–variance trade-off

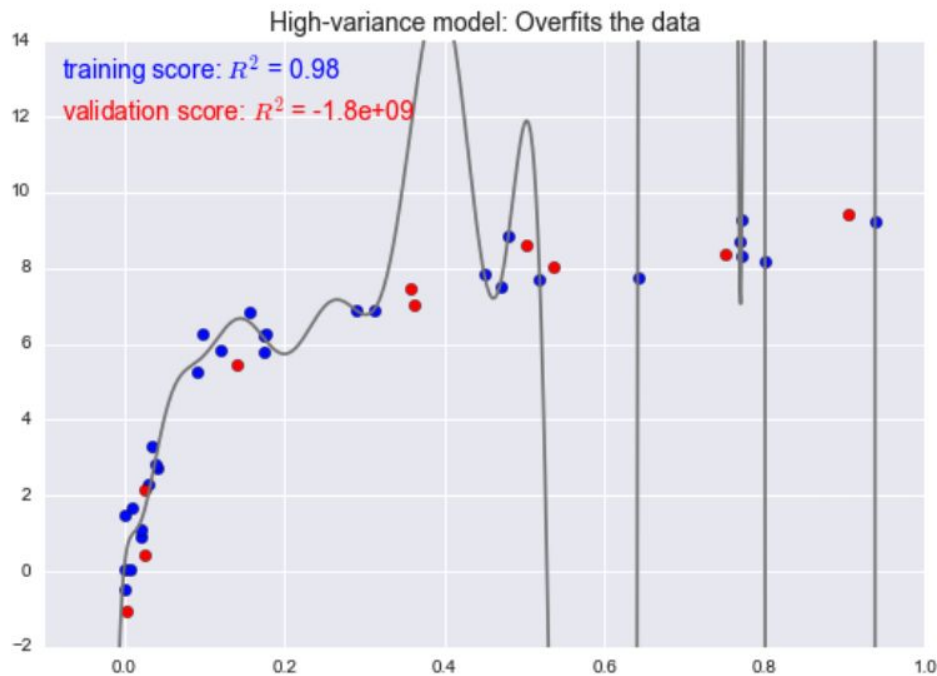
- the data are complicated, the straight-line model cannot describe this dataset well
- the model is said to underfit the data or the model has high bias.



Selecting the Best Model

The bias–variance trade-off

- the model nearly perfectly fits the training data but works worse on the validation set
- the model is said to overfit the data or the model has high variance.



R^2 score measures how well a model performs relative to a simple mean of the target values

Selecting the Best Model in Practice: Grid Search

Scikit-Learn provides automated tools to validate the model in the `grid_search` module

```
from sklearn import svm, datasets
from sklearn.model_selection import GridSearchCV

iris = datasets.load_iris()
parameters = {'kernel': ('linear', 'rbf'), 'C': [1, 10]}
svc = svm.SVC()
clf = GridSearchCV(svc, parameters)
clf.fit(iris.data, iris.target)
sorted(clf.cv_results_.keys())
```

Feature engineering

- Categorical features
- Text features
- Image features
- Derived features

Feature engineering

- Feature engineering:
 - take information you have about your problem and turn it into numbers that you can use to build your feature matrix.
 - Feature engineering on 4 common data:
 - categorical data
 - textual data
 - images
 - derived data

Categorical Features

```
data = [
```

```
    {'price': 850000, 'rooms': 4, 'neighborhood': 'Queen Anne'},
```

```
    {'price': 700000, 'rooms': 3, 'neighborhood': 'Fremont'},
```

```
    {'price': 650000, 'rooms': 3, 'neighborhood': 'Wallingford'},
```

```
    {'price': 600000, 'rooms': 2, 'neighborhood': 'Fremont'}]
```

- **numerical mapping:** the package's models make the fundamental assumption that numerical features reflect algebraic quantities => not make much sense

```
{'Queen Anne': 1, 'Fremont': 2, 'Wallingford': 3}
```

Categorical Features

```
data = [  
    {'price': 850000, 'rooms': 4, 'neighborhood': 'Queen Anne'},  
    {'price': 700000, 'rooms': 3, 'neighborhood': 'Fremont'},  
    {'price': 650000, 'rooms': 3, 'neighborhood': 'Wallingford'},  
    {'price': 600000, 'rooms': 2, 'neighborhood': 'Fremont'}]
```

- **one-hot encoding:** create extra columns indicating the presence or absence of a category with a value of 1 or 0, respectively

```
from sklearn.feature_extraction import DictVectorizer
```

```
vec = DictVectorizer(sparse=False, dtype=int)
```

```
vec.fit_transform(data)
```

```
array([[ 0,  1,  0, 850000, 4],  
       [ 1,  0,  0, 700000, 3],  
       [ 0,  0,  1, 650000, 3],  
       [ 1,  0,  0, 600000, 2]], dtype=int64)
```

Text Features

- word counts: take each snippet of text, count the occurrences of each word within it, and put the results in a table.
 - The result is a sparse matrix recording the number of times each word appears

```
from sklearn.feature_extraction.text import CountVectorizer
import pandas as pd
sample = ['problem of evil', 'evil queen', 'horizon problem']
vec = CountVectorizer()
X = vec.fit_transform(sample)
pd.DataFrame(X.toarray(), columns=vec.get_feature_names())
```

Out[8]:	evil	horizon	of	problem	queen
0	1	0	1	1	0
1	1	0	0	0	1
2	0	1	0	1	0

Text Features

- Term frequency–inverse document frequency (TF–IDF), which weights the word counts by a measure of how often they appear in the documents.

```
from sklearn.feature_extraction.text import TfidfVectorizer vec = TfidfVectorizer()  
X = vec.fit_transform(sample)  
pd.DataFrame(X.toarray(), columns=vec.get_feature_names())
```

```
Out[9]:
```

	evil	horizon	of	problem	queen
0	0.517856	0.000000	0.680919	0.517856	0.000000
1	0.605349	0.000000	0.000000	0.000000	0.795961
2	0.000000	0.795961	0.000000	0.605349	0.000000

Image Features

1. Low-Level Features (hand-crafted)
 - Extracted directly from pixel values.
2. High-Level Features (deep learning)
 - Learned automatically by neural networks (CNNs).

Derived Features

Another useful type of feature is one that is mathematically derived from some input features.

For example,

Input feature: x

Simple derived features: x^2 , x^3

Supervised learning

Naive Bayes Classification

Linear Regression

Support Vector Machines

Decision Trees

Ensemble learning methods

Naive Bayes Classification

Naive Bayes classifiers rely on Bayes theorem.

We find the probability of a label given some observed features $P(L|\text{features})$

$$P(L \mid \text{features}) = \frac{P(\text{features} \mid L)P(L)}{P(\text{features})}$$

Naïve Bayes Classification

$$c_{NB} = \operatorname{argmax}_{c_j \in C} P(c_j) \prod_{i \in \text{positions}} P(w_i | c_j)$$

$$P(c_j) \leftarrow \frac{|docs_j|}{|\text{total \# documents}|}$$

$$P(w_k | c_j) \leftarrow \frac{n_k + \alpha}{n + \alpha |Vocabulary|}$$

Let's do a worked sentiment example!

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

A worked sentiment example with add-1 smoothing

	Cat	Documents
Training	-	just plain boring
	-	entirely predictable and lacks energy
	-	no surprises and very few laughs
	+	very powerful
	+	the most fun film of the summer
Test	?	predictable with no fun

1. Prior from training:

$$\hat{P}(c_j) = \frac{N_{c_j}}{N_{total}} \quad \begin{array}{l} P(-) = 3/5 \\ P(+) = 2/5 \end{array}$$

2. Drop "with"

3. Likelihoods from training:

$$p(w_i|c) = \frac{\text{count}(w_i, c) + 1}{(\sum_{w \in V} \text{count}(w, c)) + |V|}$$

$$P(\text{"predictable"}|-) = \frac{1+1}{14+20} \quad P(\text{"predictable"}|+) = \frac{0+1}{9+20}$$

$$P(\text{"no"}|-) = \frac{1+1}{14+20} \quad P(\text{"no"}|+) = \frac{0+1}{9+20}$$

$$P(\text{"fun"}|-) = \frac{0+1}{14+20} \quad P(\text{"fun"}|+) = \frac{1+1}{9+20}$$

4. Scoring the test set:

$$P(-)P(S|-) = \frac{3}{5} \times \frac{2 \times 2 \times 1}{34^3} = 6.1 \times 10^{-5}$$

$$P(+)P(S|+) = \frac{2}{5} \times \frac{1 \times 1 \times 2}{29^3} = 3.2 \times 10^{-5}$$

Naive Bayes Classification

Advantages:

- They are extremely fast for both training and prediction
- They provide straightforward probabilistic prediction
- They are often very easily interpretable
- They have very few (if any) tunable parameters

=> a good choice as an initial baseline classification

Supervised learning

Naive Bayes Classification

Linear Regression

Support Vector Machines

Decision Trees

Ensemble learning methods

Linear Regression

- Simple Linear Regression
- Basis Function Regression
- Regularization

Simple Linear Regression

- The most familiar linear regression, a straight-line fit to data.
- A straight-line fit is a model of the form $y = \beta x + \alpha$ where β is commonly known as the *slope*, and α is commonly known as the *intercept*.
- Objective: estimate the slope β and the intercept α

Simple Linear Regression

Assuming we've determined such an alpha and beta, then we make predictions simply with:

```
def predict(alpha, beta, x_i):  
    return beta * x_i + alpha
```

We can compute the error for each pair:

```
def error(alpha, beta, x_i, y_i):  
    return y_i - predict(alpha, beta, x_i)
```

We add up the squared errors:

```
def sum_of_squared_errors(alpha, beta, x, y):  
    return sum(error(alpha, beta, x_i, y_i) ** 2 for x_i, y_i in zip(x, y))
```

The least squares solution is to choose the alpha and beta that make `sum_of_squared_errors` as small as possible, the alpha and beta are given by:

```
def least_squares_fit(x, y):  
    beta = correlation(x, y) * standard_deviation(y) / standard_deviation(x)  
    alpha = mean(y) - beta * mean(x)  
    return alpha, beta
```

Simple Linear Regression

Figure out how well we've fit the data

```
def de_mean(xs):
```

```
    x_bar = mean(xs)
```

```
    return [x - x_bar for x in xs]
```

```
def total_sum_of_squares(y):
```

```
    return sum(v ** 2 for v in de_mean(y))
```

```
def r_squared(alpha, beta, x, y):
```

```
    #The higher R2 is, the better our model fits the data
```

```
    return 1.0 - (sum_of_squared_errors(alpha, beta, x, y) / total_sum_of_squares(y))
```

Simple Linear Regression

Data for linear regression

```
rng = np.random.RandomState(1)
x = 10 * rng.rand(50)
y = 2*x-5+rng.randn(50)
plt.scatter(x, y)
```

Scikit-Learn's LinearRegression estimator is used to fit this data and construct the best-fit line

```
from sklearn.linear_model import LinearRegression
model = LinearRegression(fit_intercept=True)
model.fit(x[:, np.newaxis], y)
xfit = np.linspace(0, 10, 1000)
yfit = model.predict(xfit[:, np.newaxis])
plt.scatter(x, y)
plt.plot(xfit, yfit)
```

Simple Linear Regression

- LinearRegression can handle multidimensional linear models of the form:

$$y = a_0 + a_1x_1 + a_2x_2 + \dots$$

where there are multiple x values.

- This is similar to fitting a plane to points in three dimensions, or fitting a hyper-plane to points in higher dimensions.

Linear Regression

- Simple Linear Regression
- Basis Function Regression
- Regularization

Basis Function Regression

Add complexity to a model by considering nonlinear features of the input data

- Polynomial basis functions

Basis Function Regression

Polynomial basis functions

```
import numpy as np
from sklearn.preprocessing import PolynomialFeatures
X = np.arange(6).reshape(3, 2)
poly = PolynomialFeatures(2)
poly.fit_transform(X)
```

The features of X have been transformed from (X1, X2) to (1, X1, X2, X1¹, X1X2, X2²)

X

```
array([[0, 1],
       [2, 3],
       [4, 5]])
```

```
poly.fit_transform(X)
array([[ 1.,  0.,  1.,  0.,  0.,  1.],
       [ 1.,  2.,  3.,  4.,  6.,  9.],
       [ 1.,  4.,  5., 16., 20., 25.]])
```

Linear Regression

- Simple Linear Regression
- Basis Function Regression
- Regularization

Regularization

Penalizing large values of the model parameters ([regulation.py](#))

- Ridge regression (L2 regularization) or Tikhonov regularization penalizing the sum of squares (2-norms) of the model coefficients

$$P = \alpha \sum_{n=1}^N \theta_n^2$$

where α is a free parameter that controls the strength of the penalty

- Lasso regularization (L1) penalizing the sum of absolute values (1-norms) of regression coefficients

$$P = \alpha \sum_{n=1}^N |\theta_n|$$

Supervised learning

Naive Bayes Classification

Linear Regression

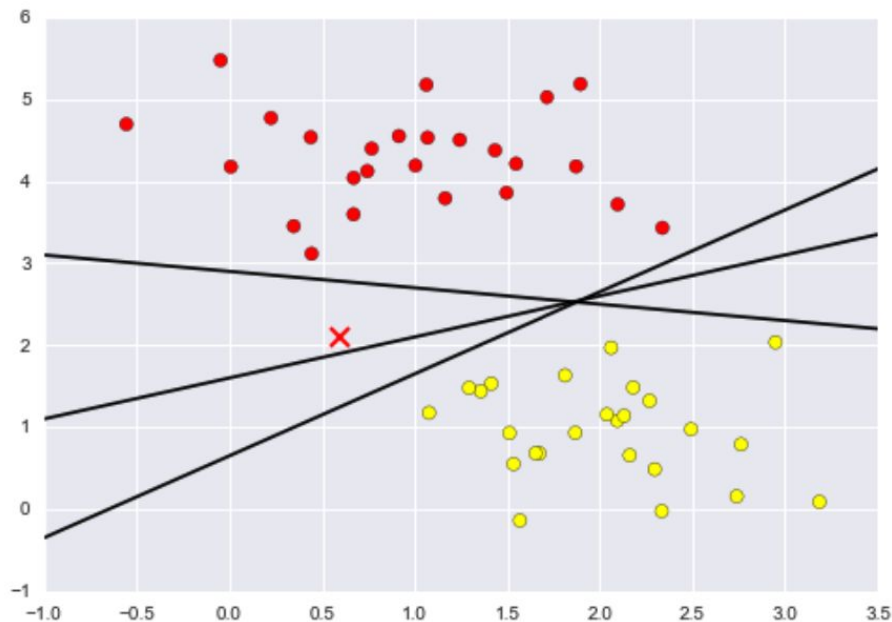
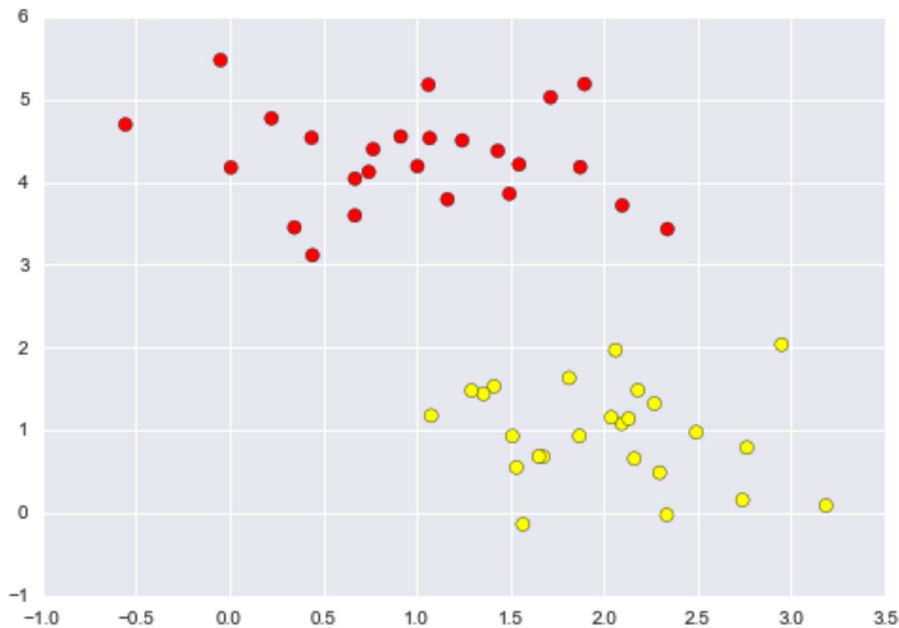
Support Vector Machines

Decision Trees

Ensemble learning methods

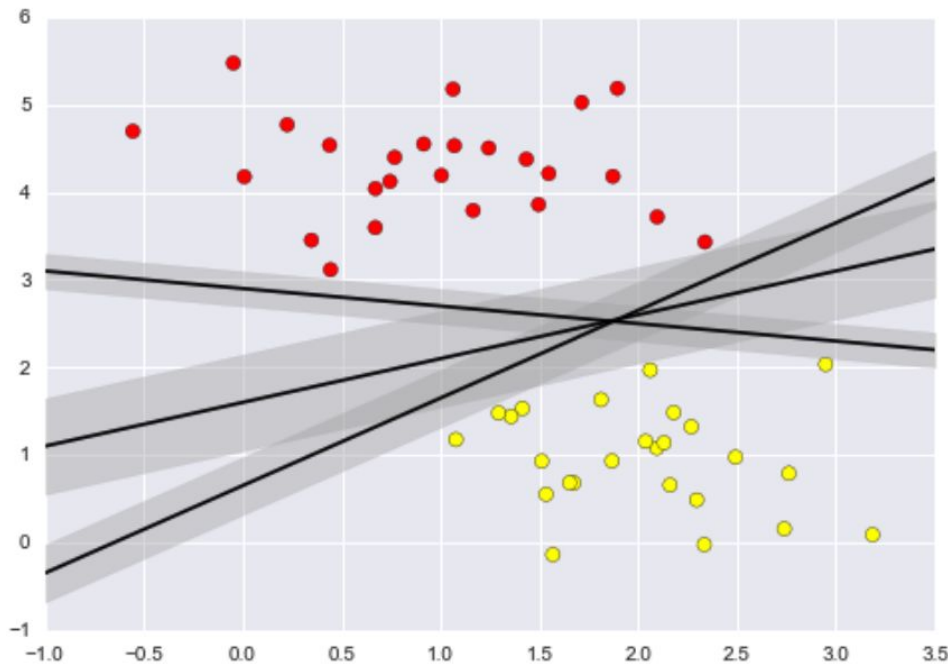
Support Vector Machines

- A linear discriminative classifier would attempt to draw a straight line separating the two sets of data, and thereby create a model for classification.
- Problem: there is more than one possible dividing line that can perfectly discriminate between the two classes!



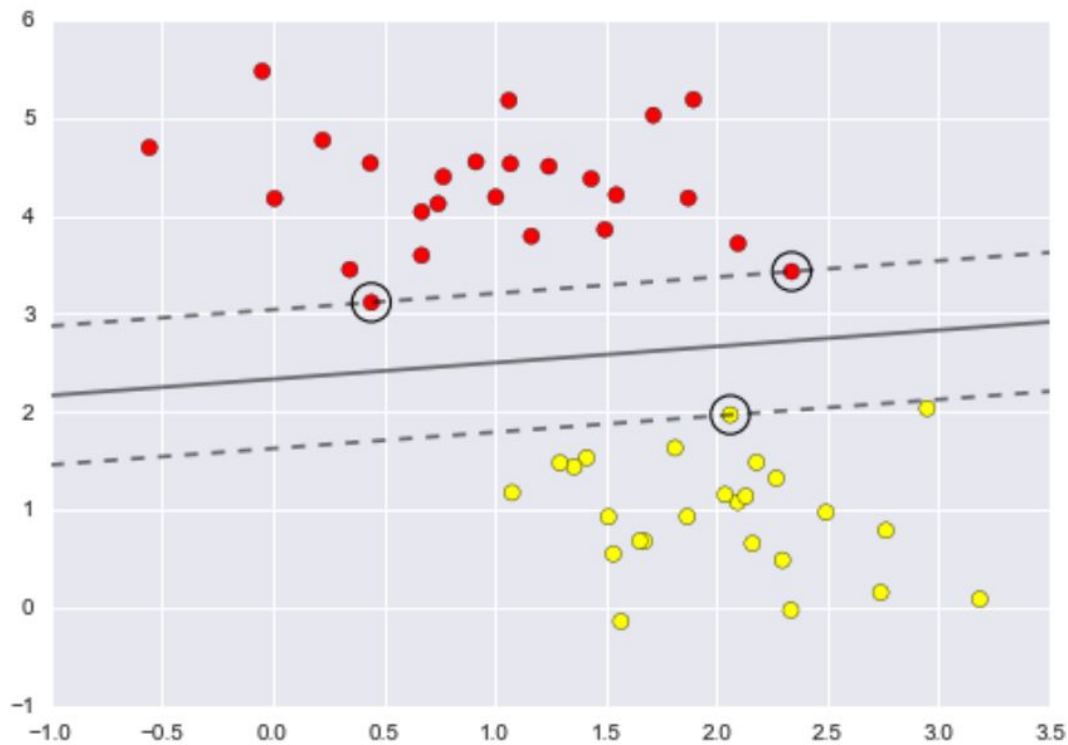
Support Vector Machines

- Rather than simply drawing a zero-width line between the classes, SVM draws around each line a margin of some width, up to the nearest point.
- The line that maximizes this margin is the one we will choose as the optimal model.



Support Vector Machines

SVM finds the dividing line that maximizes the *margin* between the two sets of points. A few of the training points just touch the margin, they are known as the *support vectors*.



Support Vector Machines

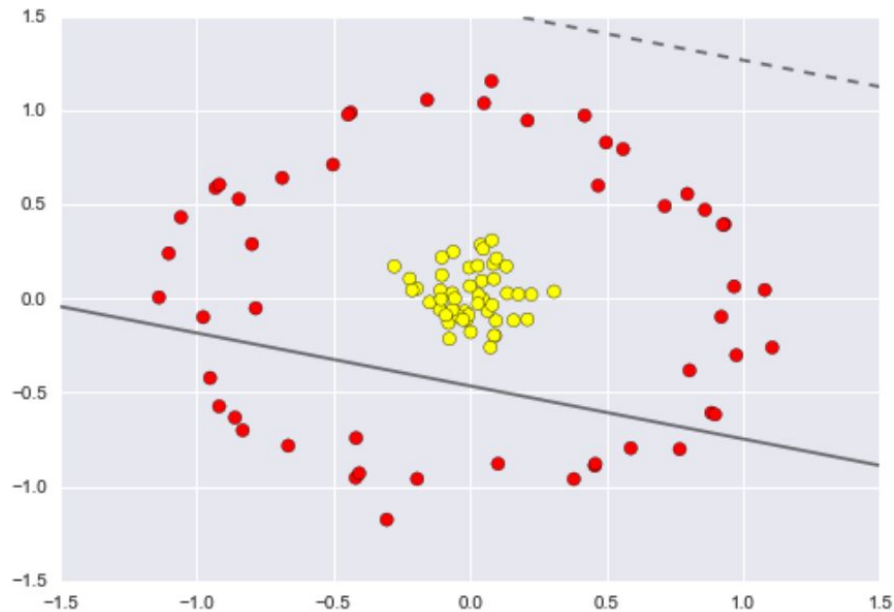
```
from sklearn import svm
from sklearn.datasets import make_blobs
# we create 40 separable points
X, y = make_blobs(n_samples=40, centers=2, random_state=6)
# fit the model, don't regularize for illustration purposes
clf = svm.SVC(kernel="linear", C=1000)
clf.fit(X, y)
# get support vectors
print(clf.support_vectors_)
# get indices of support vectors
print(clf.support_)
# get number of support vectors for each class
print(clf.n_support_)
```

Support Vector Machines

Beyond linear boundaries: Kernel transformation

Let's look at some data that is not linearly separable.

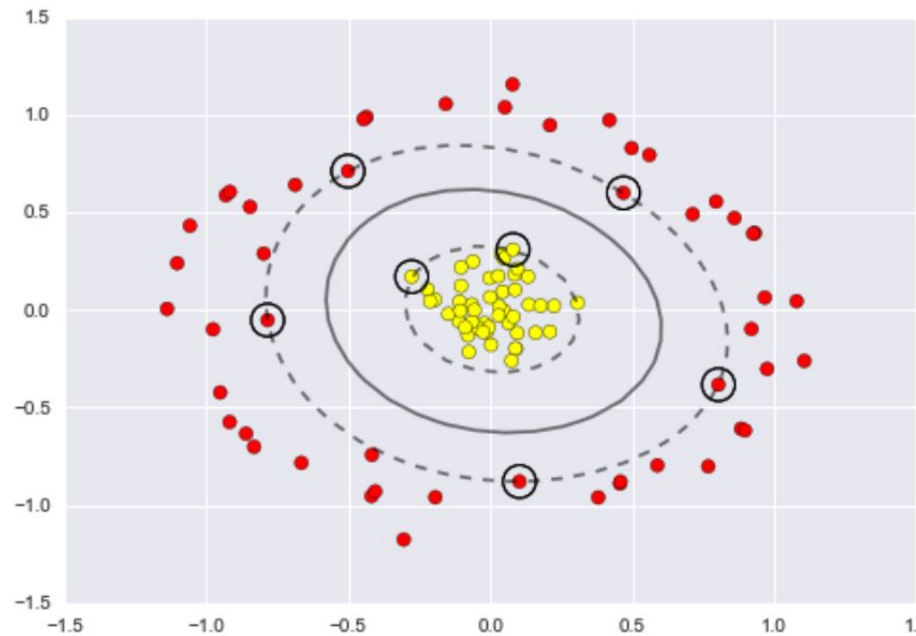
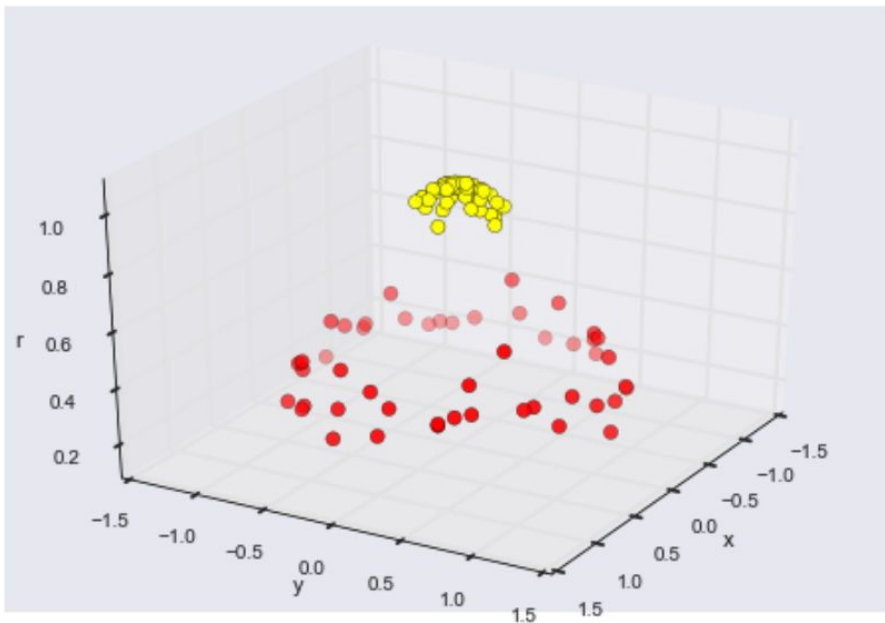
It is clear that no linear discrimination will ever be able to separate this data.



Support Vector Machines

Beyond linear boundaries: Kernel transformation

- We might project the data into a higher dimension such that a linear separator would be sufficient. Radial basis function : $r = \exp(-(X^2).sum(1))$
- With this additional dimension, the data becomes trivially linearly separable.



Support Vector Machines

```
from sklearn import svm

from sklearn.datasets import make_circles

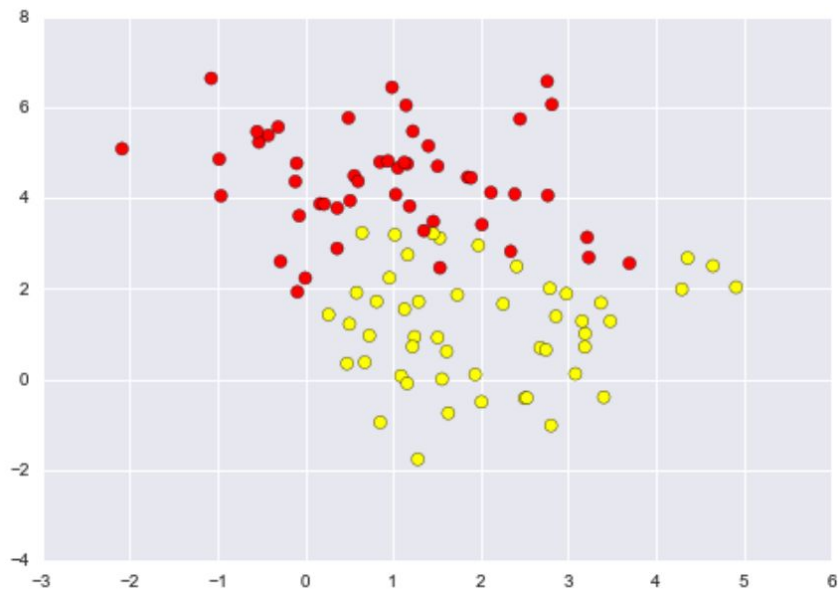
# we create 40 separable points
X, y = make_circles(100, factor=.1, noise=.1)

# fit the model, don't regularize for illustration purposes
clf = svm.SVC(kernel="rbf", C=1000)
clf.fit(X, y)
```

Support Vector Machines

Tuning the SVM: Softening margins

If data has some amount of overlap, how to deal with it?



Support Vector Machines

Tuning the SVM: Softening margins

- SVM implementation “softens” the margin; that is, it allows some of the points to creep into the margin if that allows a better fit.
- The hardness of the margin is controlled by a tuning parameter, most often known as C .
 - For very large C , the margin is hard, and points cannot lie in it.
 - For smaller C , the margin is softer, and can grow to encompass some points.

Support Vector Machines

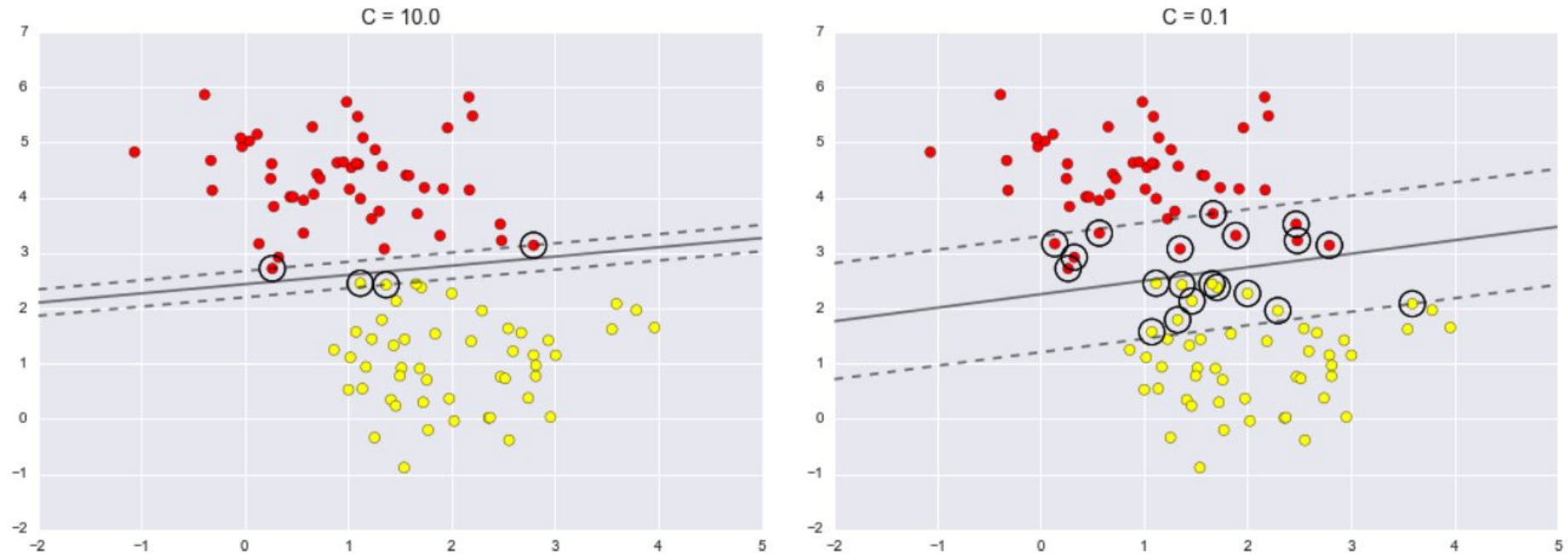


Figure 5-63. The effect of the C parameter on the support vector fit

Support Vector Machine

Advantages:

- They are very compact models, and take up very little memory.
- Once the model is trained, the prediction phase is very fast.
- They work well with high-dimensional data
- Their integration with kernel methods makes them easy to adapt to many types of data.

Support Vector Machine

Disadvantages:

- For large numbers of training samples, this computational cost can be prohibitive.
- The results are strongly dependent on a suitable choice for the softening parameter C .
- The results do not have a direct probabilistic interpretation. This can be estimated via an internal cross-validation (see the probability parameter of SVC), but this extra estimation is costly.

Supervised learning

Naive Bayes Classification

Linear Regression

Support Vector Machines

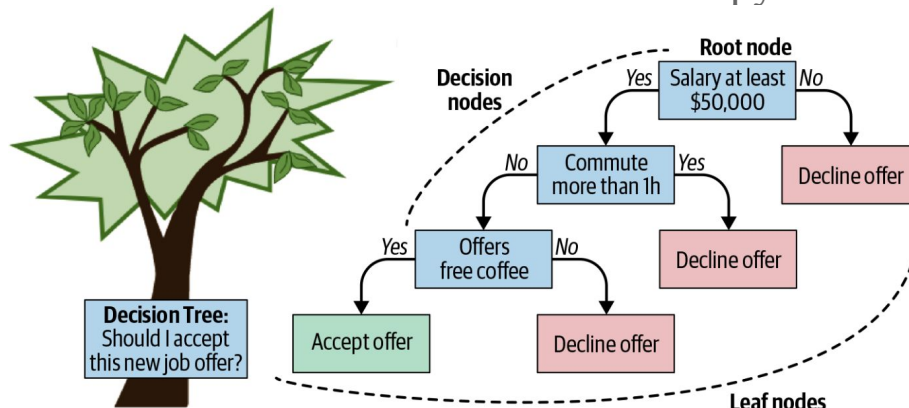
Decision Trees

Ensemble learning methods

Decision Trees and Random Forests

Decision Trees

- A decision tree uses a tree structure to represent a number of possible decision paths and an outcome for each path.
- In order to build a decision tree, we will need to decide what questions to ask and in what order.
- Ideally, we'd like to choose questions whose answers give a lot of information about what our tree should predict.
- We capture this notion of “how much information” with entropy.



Decision Trees and Random Forests

Decision Trees: ID3 Algorithm

1. Choose A as the “best” decision attribute for next node based on Entropy.
2. Assign A as decision attribute for node.
3. For each value of A , create new descendant.
4. Splitting training examples to leaf node according to the value of attribute A .
5. If all training examples are perfectly classified (same value of target attribute) then stop, else iterate over new leaf nodes.

Decision Trees and Random Forests

```
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()
X= iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

clf = DecisionTreeClassifier(random_state=0)
clf = clf.fit(X_train,y_train)
y_pred = clf.predict(X_test)
```

Supervised learning

Naive Bayes Classification

Linear Regression

Support Vector Machines

Decision Trees

Ensemble learning methods

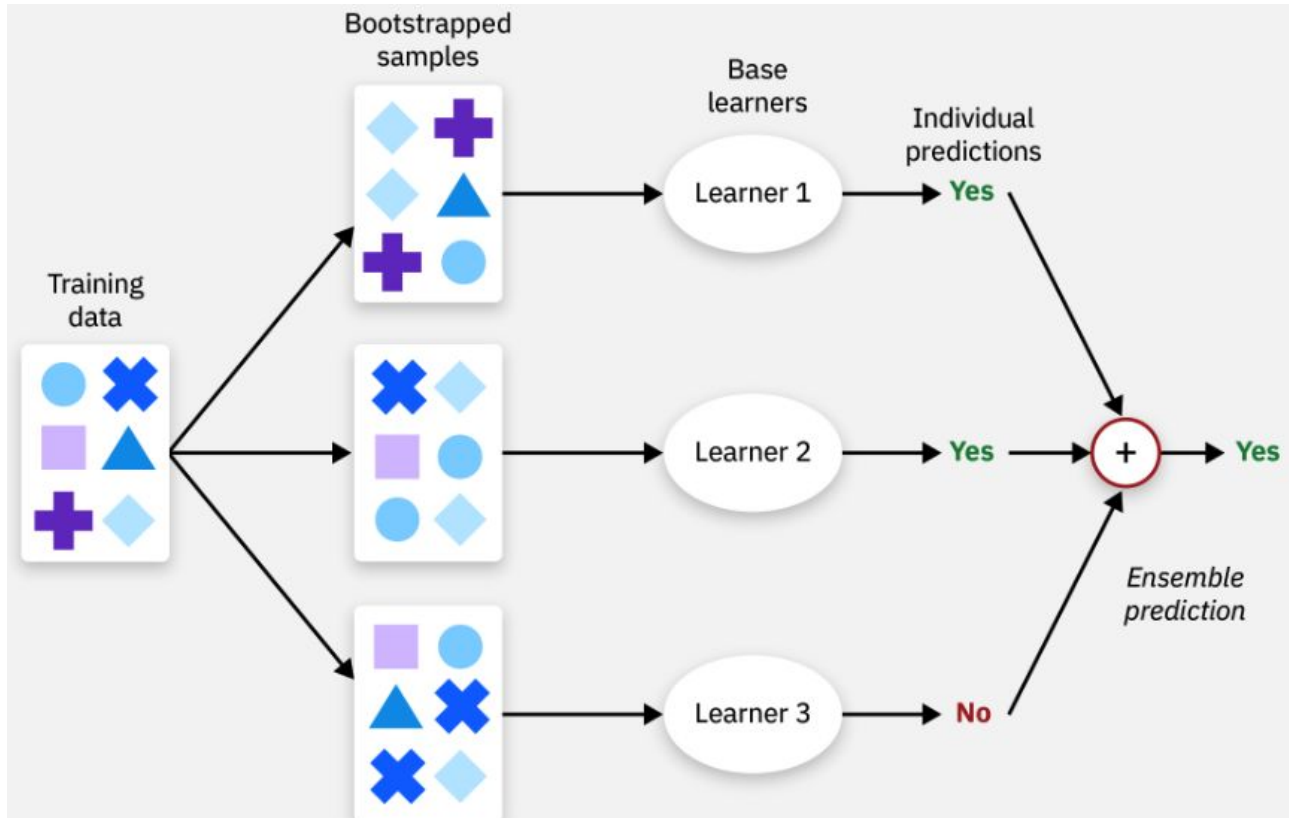
Ensemble learning methods

- Ensemble learning methods are made up of a set of classifiers and their predictions are aggregated to identify the most popular result.
- 3 popular methods:
 - Bagging
 - Stacking
 - Boosting

Ensemble learning methods - Bagging

- Data: bootstrapped samples
 - A random sample of data in a training set is selected with replacement
- Train:
 - After several data samples are generated, these models are then trained independently.
 - Depending on the type of task the average or majority of those predictions yield a more accurate estimate.

Ensemble learning methods - Bagging



Ensemble learning methods - Bagging

```
from sklearn.svm import SVC
from sklearn.ensemble import BaggingClassifier
from sklearn.datasets import make_classification
X, y = make_classification(n_samples=100, n_features=4,
                          n_informative=2, n_redundant=0,
                          random_state=0, shuffle=False)
clf = BaggingClassifier(estimator=SVC(),
                       n_estimators=10, random_state=0).fit(X, y)
clf.predict([[0, 0, 0, 0]])
```

Ensemble learning methods - Bagging

- Random forest is an extension of the bagging method
- Data:
 - From a training set, random data samples are selected with replacement in order to create training sample
 - From the training sample, one-third of it is set aside as test data, known as the out-of-bag (oob) sample
- Feature bagging:
 - random subsets of features are generated
- Train:
 - Models are then trained independently on random data samples with random subsets of features
 - For a regression task, the individual decision trees will be averaged
 - For a classification task, a majority vote will yield the predicted class.
 - Finally, the oob sample is then used for cross-validation, finalizing that prediction.

Ensemble learning methods - Bagging

```
from sklearn import datasets

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier


iris = datasets.load_iris()

X= iris.data

y = iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3) # 70% training and 30% test

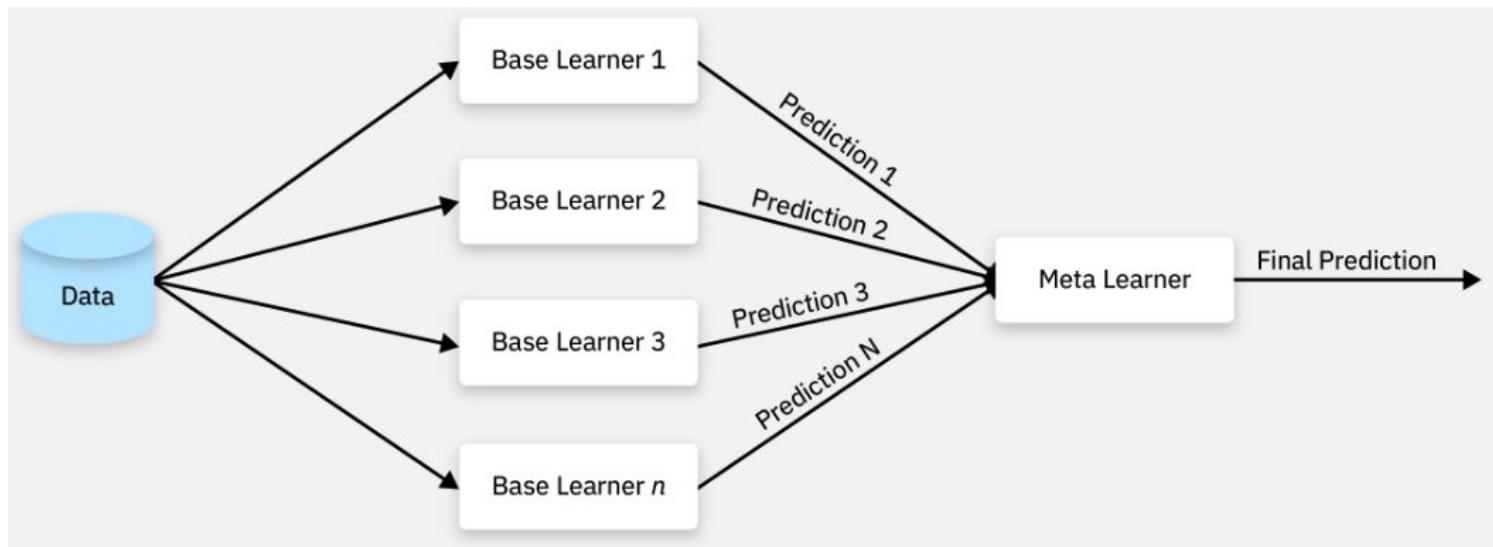

clf=RandomForestClassifier(n_estimators=100, max_depth=15, )

clf.fit(X_train,y_train)

y_pred=clf.predict(X_test)
```

Ensemble learning methods - Stacking

- Stacking specifically trains several base learners from the same dataset using a different training algorithm for each learner.
- Each base learner makes predictions on an unseen dataset.
- These first model predictions are then compiled and used to train a final model, being the meta-model.



Ensemble learning methods - Stacking

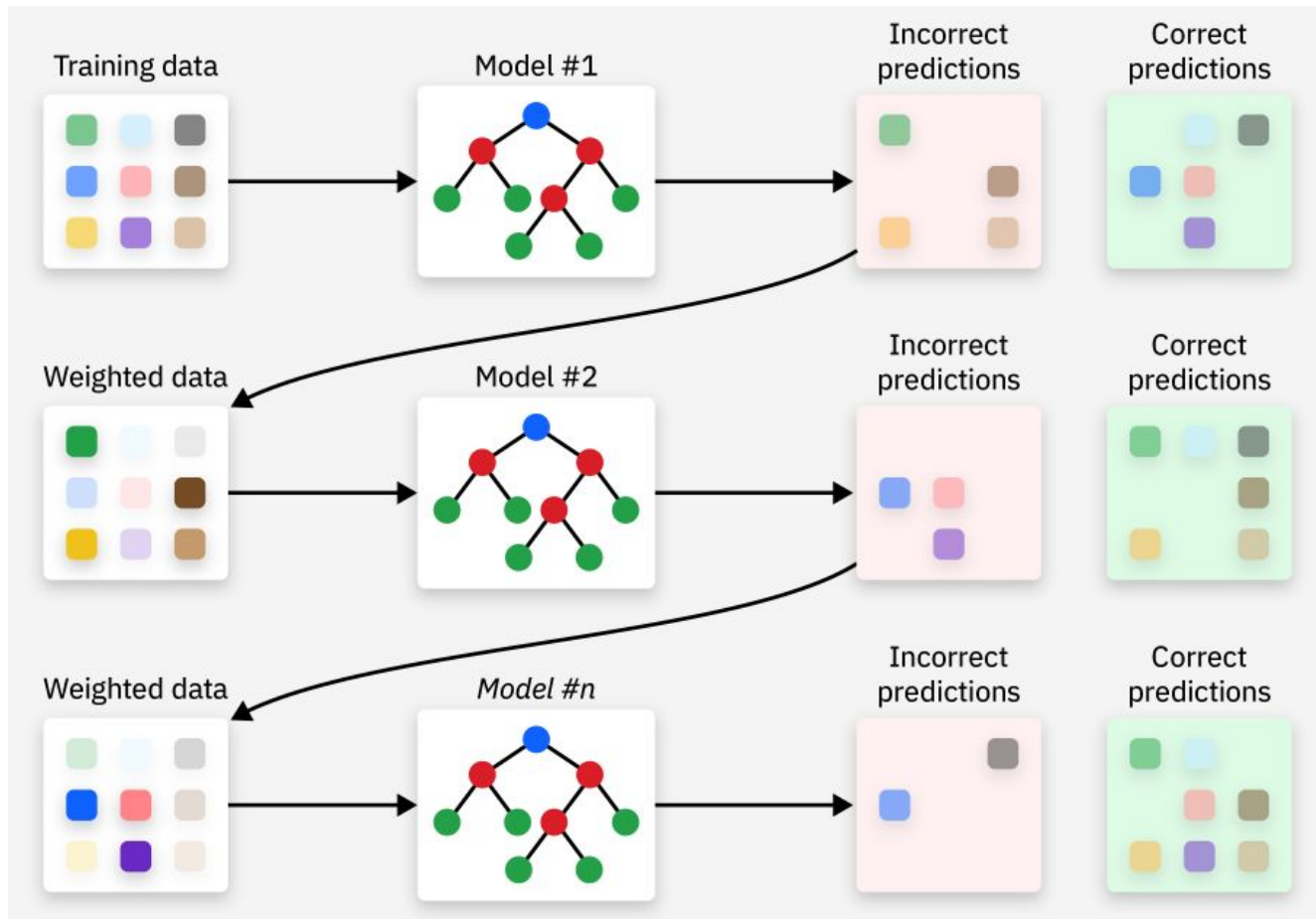
```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier, StackingClassifier
from sklearn.svm import LinearSVC
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

X, y = load_iris(return_X_y=True)
estimators = [
    ('rf', RandomForestClassifier(n_estimators=10, random_state=42)),
    ('svr', make_pipeline(StandardScaler(),
                          LinearSVC(random_state=42)))
]
clf = StackingClassifier(
    estimators=estimators, final_estimator=LogisticRegression()
)
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, stratify=y, random_state=42
)
clf.fit(X_train, y_train).score(X_test, y_test)
```

Ensemble learning methods - Boosting

- Boosting
 - Data:
 - The training set used for each classifier is chosen based on the performance of the earlier classifier(s).
 - Samples that are incorrectly predicted by previous classifiers in the series are chosen more often than examples that were correctly predicted.
 - Train:
 - Weak learners are trained sequentially
 - Boosting method attempts to produce new classifiers that are better able to predict samples for which the current ensemble's performance is poor.

Ensemble learning methods - Boosting



Ensemble learning methods - Boosting

- Two popular boosting methods
 - Adaptive boosting: when creating a new iteration of a dataset for training the next learner, AdaBoost adds weights to the previous learner's misclassified samples, causing the next learner to prioritize those misclassified samples.
 - Gradient boosting uses residual errors from a previous model to set target predictions for the next model.

Ensemble learning methods - Boosting

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.datasets import make_classification
X, y = make_classification(n_samples=1000, n_features=4,
                          n_informative=2, n_redundant=0,
                          random_state=0, shuffle=False)
clf = AdaBoostClassifier(n_estimators=100, algorithm="SAMME", random_state=0)
clf.fit(X, y)
clf.predict([[0, 0, 0, 0]])
```

Ensemble learning methods - Boosting

```
from sklearn import datasets  
from sklearn.model_selection import train_test_split  
from sklearn.ensemble import GradientBoostingClassifier  
  
iris = datasets.load_iris()  
X= iris.data  
y = iris.target  
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)  
clf=GradientBoostingClassifier()  
clf.fit(X_train,y_train)  
y_pred=clf.predict(X_test)
```

Unsupervised learning

Principal Component Analysis

k-Means Clustering

Principal Component Analysis

Principal Components Analysis: a fast and flexible unsupervised method for dimensionality reduction

Suppose that the data to be reduced consist of tuples or data vectors described by n attributes or dimensions.

- PCA searches for k n -dimensional orthogonal vectors that can best be used to represent the data, where $k \leq n$.
- PCA “combines” the essence of attributes by creating an alternative, smaller set of variables

Principal Component Analysis

```
from sklearn import datasets, decomposition
```

```
iris = datasets.load_iris()
```

```
X = iris.data
```

```
pca = decomposition.PCA(n_components=3)
```

```
pca.fit(X)
```

```
X = pca.transform(X)
```

k-Means Clustering

The k-means algorithm searches for a predetermined number of clusters within an unlabeled multidimensional dataset.

Two assumptions:

- The “cluster center” is the arithmetic mean of all the points belonging to the cluster.
- Each point is closer to its own cluster center than to other cluster centers.

k-Means Clustering

K-means involves an intuitive iterative approach known as expectation–maximization.

1. Guess some cluster centers
2. Repeat until converged
 - a. E-Step: assign points to the nearest cluster center
 - b. M-Step: set the cluster centers to the mean

Under typical circumstances, each repetition of the E-step and M-step will always result in a better estimate of the cluster characteristics.